

Übung 4: Objekte, einfache Datenstrukturen, var und Enums

Aufgabe 0: String-Vergleich	1
Aufgabe 1: Eigener Stack mit Array	2
Aufgabe 2: Korrekte Klammerungen	3
Aufgabe 3: Eigene Liste	4
Aufgabe 4: Coffee Shop Enum	5

Lernziele

- Sie können den Typ eines Objekts bestimmen und zwei Objekte auf inhaltliche Gleichheit prüfen.
- Sie können mit Klassen eigene Datenstrukturen implementieren.
- Sie kennen var als Schreibweisenabkürzung für längere Typnamen.
- Sie kennen die Verwendungsgebiete von Aufzählungen (enums).

Hinweis: Versuchen Sie, ab dieser Serie, wo sinnvoll, `var`-Deklarationen einzusetzen.

Aufgabe 0: String-Vergleich

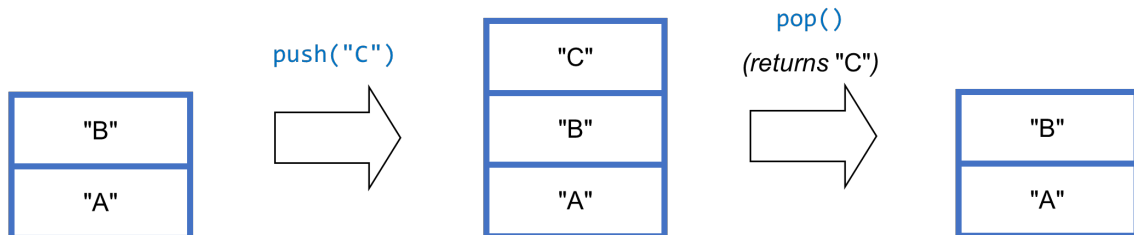
In Java ist es syntaktisch korrekt, Objekte mit `==` und `equals()` zu vergleichen. Dies muss aber nicht zwingend das Gleiche bedeuten. In der Vorlage sind diese Operationen auf unterschiedlich erstellte Objekte angewendet. Das Resultat des Vergleichs wird jeweils ausgegeben.

- Analysieren Sie den Code und die Ausgabe
- Welche Konsequenzen ergeben sich aus der beobachteten Ausgabe? Formulieren Sie für sich, welche Schlüsse Sie aus der beobachteten Ausgabe ziehen können.

Aufgabe 1: Eigener Stack mit Array

Entwickeln Sie eine eigene Stack-Datenstruktur als Klasse namens `OwnStack`.

Der Stack befolgt das **LIFO**-Prinzip („Last In First Out“): Man kann jeweils ein Element zuoberst auf den Stack legen (Operation `push()`) und jeweils das oberste Element vom Stack entfernen (Operation `pop()`).



Unser Stack hat eine maximale Kapazität und soll nur **Strings** als Elemente unterstützen. Benutzen Sie intern ein **Array** zur Speicherung.

Konkret soll die Klasse `OwnStack` folgende Funktionalitäten anbieten:

- `Stack(int capacity)`
 - Konstruktor: Initialisiert den Stack mit einer Kapazität (maximale Anzahl Elemente).
- `void push(String element)`
 - Legt einen `String` zuoberst auf den Stack. Wenn der Stack voll ist, darf es einen Fehler geben.
- `String pop()`
 - Nimmt den obersten `String` vom Stack und gibt diesen zurück. Wenn der Stack leer ist, wird `null` zurückgegeben.
- `int size()`
 - Liefert die Anzahl der Elemente auf dem Stack.
- `boolean isEmpty()`
 - `true`, falls der Stack leer ist.
- `boolean isFull()`
 - `true`, falls die Stack voll ist, d.h. Kapazität erreicht ist.

Die Vorlage enthält bereits Testfälle, mit denen Sie Ihren Stack überprüfen können.

Aufgabe 2: Korrekte Klammerungen

Es soll die zuvor in Aufgabe 1 entwickelte **Stack-Klasse** namens `OwnStack` benutzt werden, um korrekt geschachtelte Klammerungen in einem Text zu erkennen (z. B. XML, mathematische Ausdrücke). Eine Klammerung ist korrekt, wenn auf jede öffnende Klammer (runde, eckige, geschweifte, spitze) später eine entsprechende schliessende Klammer folgt. Dabei kann innerhalb der öffnenden und schliessenden Klammer wieder eine geschachtelte korrekte Klammerung enthalten sein.

Beispiele korrekter Klammerungen:

- `( < [ ( ) ] > ) { }`
- `( ) [ ] { } < >`
- `( ( ( < > )))( [ < > ] )`
- `""` (*leerer String*)

Beispiele fehlerhafter Klammerungen:

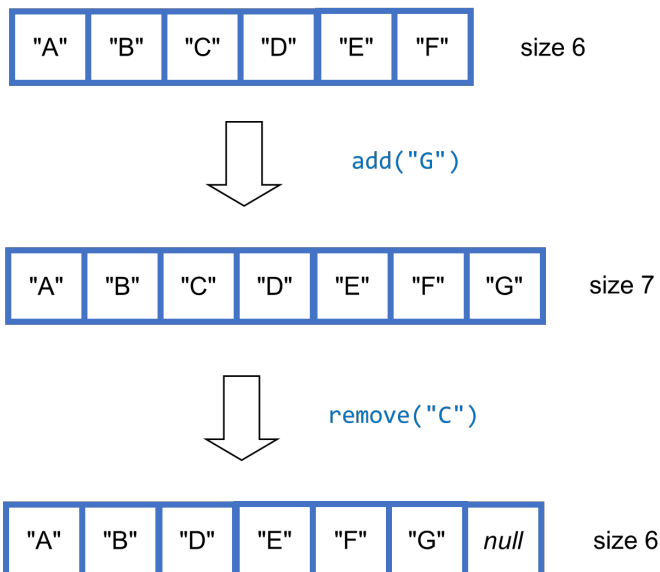
- `<`
- `[ ( ) >`
- `{ )( ] [ }`

Tipp: Mittels `String.toCharArray()` erhalten Sie die einzelnen Buchstaben eines Texts als Array.

Aufgabe 3: Eigene Liste

Wenn man beliebig viele Elemente speichern möchte, sind reine Arrays unpraktisch, da sie eine fixe Länge haben. Deshalb entwickeln wir eine eigene Klasse `MyArrayBasedList`, welche eine beliebig lange Abfolge von Elementen speichern kann.

Intern soll die Liste ein **Array** benutzen. Wenn der Platz ausgeht, wird das Array in ein grösseres umkopiert. (Verwenden Sie hier nur Arrays und keine bereits vorgefertigten Java-Listen!)



Die Liste unterstützt beliebige Referenztypen als Elemente: Dazu ist der Typ `Object` nützlich.

- `void add(Object element)`
 - Füge ein Element zuhinterst in die Liste ein.
- `Object get(int position)`
 - Gibt das Element an jener Position zurück. Die Positionen sind von 0 an durchnummeriert. Für eine ungültige Position wird `null` retourniert.
- `boolean contains(Object element)`
 - Prüft, ob ein gleiches Element in der Liste enthalten ist.
- `void remove(Object element)`
 - Entferne das erste Vorkommen von `element`. Kein Effekt, falls die Liste das Element nicht enthält.

- `int size()`
 - Gibt die Anzahl der Elemente in der Liste zurück.

Die Vorlage enthält bereits Testfälle für Ihre Implementation.

Performance-Tipp: Damit nicht bei jedem `add()` alles in ein neues Array umkopiert werden muss, kann man auch ein neues Array mit leeren Vorratsplätzen anlegen, z. B. ein Array, das 1.5 Mal grösser ist.

Aufgabe 4: Coffee Shop Enum

In der Vorlage finden Sie ein System, welches Kaffee-Bestellungen im Kaffeehaus *The Java Bean* verwalten soll. Die Klassen für die Bestellungen sind bereits vorhanden, allerdings fehlt ein `enum` um die verschiedenen Kaffeearten zu definieren.

Implementieren Sie den `enum` namens `CoffeeType`. Jeder Kaffeetyp soll die folgenden Eigenschaften haben:

- Eine Grösse (`String`): “Klein”, “Mittel”, “Gross”
- Einen `int`-Wert für die Anzahl Espresso-Shots die der Kaffee enthält.
- Ein `boolean`, der angibt, ob der Kaffee entkoffeiniert ist oder nicht.

Die obigen Eigenschaften sollen über die `public`-Methoden `getSize()`, `getAmountOfShots()` und `isDecaf()` abrufbar sein.

Es soll die folgenden vier Kaffeetypen geben:

- **ESPRESSO:** Mittel, 2 Shots, nicht entkoffeiniert.
- **LATTE:** Gross, 1 Shot, nicht entkoffeiniert.
- **AMERICANO:** Klein, 2 Shots, entkoffeiniert.
- **MOCHA:** Mittel, 1 Shot, nicht entkoffeiniert.

In der Vorlage finden Sie auch Unit Tests um Ihre `enum`-Implementation zu testen.